Data.Semigroup: https://hackage.haskell.org/package/base/docs/Data-Semigroup.html
Data.Monoid: https://hackage.haskell.org/package/base/docs/Data-Monoid.html

Data.Semigroup

```
-- Data.Semigroup
class Semigroup a where
        sconcat :: NonEmpty a -> a
        stimes :: Integral b => b -> a -> a

stimesMonoid :: (Integral b, Monoid a) => b -> a -> a
stimesIdempotent :: Integral b => b -> a -> a
stimesIdempotentMonoid :: (Integral b, Monoid a) => b ->
a -> a
mtimesDefault :: (Integral b, Monoid a) => b -> a -> a

newtype Min a = Min { getMin :: a }
newtype Max a = Max { getMax :: a }
newtype First a = First { getFirst :: a }
newtype Last a = Last { getLast :: a }
newtype WrappedMonoid m = WrapMonoid { unwrapMonoid ::
m }
newtype Dual a = Dual { getDual :: a }
newtype Endo a = Endo { appEndo :: a -> a }
newtype All = All { getAll :: Bool }
newtype Any = Any { getAny :: Bool }
newtype Sum a = Sum { getSum :: a }
newtype Product a = Product { getProduct :: a }

diff :: Semigroup m => m -> Endo m
cycle1 :: Semigroup m => m -> m

data Arg a b = Arg a b
type ArgMin a b = Min (Arg a b)
type ArgMax a b = Max (Arg a b)
```

Data.Monoid

```
-- Data.Monoid
class Semigroup a => Monoid a where
        mempty  :: a
        mappend :: a -> a -> a
        mconcat :: [a] -> a

newtype First a = First { getFirst :: Maybe a }
newtype Last a = Last { getLast :: Maybe a }
newtype Alt f a = Alt { getAlt :: f a }
newtype Ap f a = App { getApp :: f a }
```

Semigroup
Monoid

semigroup = **sum** operation
monoid = sum operation + **zero element**

(<>) :: a -> a -> a	semigroup sum
mempty :: a	monoid zero

Laws

```
   x <> (y <> z)   ==    (x <> y) <> z
 associative
   x <> mempty     ==    mempty <> x
identity
                   ==    x
```

e.g. Monoid [a]

```
instance Semigroup [a] where
  (<>) = (++)
instance Monoid [a] where
  mempty = []


[1,2,3] <> [10,20]            -- [1,2,3,10,20]
[1,2,3] <> mempty             -- [1,2,3]
```

Monoid Wrappers

Monoid wrappers

For a given type, there may be **multiple** ways to define a monoid.
e.g. Int monoid = + or * or -
A monoid wrapper is a wrapper implementing a specific monoid definition

Dual	<> = reverse order i.e. Dual x <> Dual y = Dual (y <> x)
Endo	<> = function composition i.e. Endo f <> Endo g = Endo (g . f)
All	<> = &&
Any	<> = ||
Sum	<> = +
Product	<> = *
First	<> = first Just
Last	<> = last Just i.e. First (Just x) <> First (Just y) = First (Just x) Last (Just x) <> Last (Just y) = Last (Just y)
Alt	<> = sum applicative by <|>
Ap	<> = sum applicative by inner type i.e. Alt x <> Alt y = Alt (x <|> y) Ap x <> Ap y = Ap ((<>) <$> x <*> y)

Example: fold same value with different monoids

```
import Data.Monoid

getSum $ foldMap Sum [1..10]                    -- 55
```

```
getProduct $ foldMap Product [1..10]            --
 3628800
(appEndo $ foldMap (Endo . (+)) [1..10]) 0      -- 55
(appEndo $ foldMap (Endo . (*)) [1..10]) 1      --
3628800

getAll $ foldMap All [True, False, True]        -- False
getAll $ foldMap Any [True, False, True]        -- True

getFirst $ foldMap First [Just 1, Just 2]       -- Just
1
getFirst $ foldMap First [Nothing, Just 2]      -- Just
2
getFirst $ foldMap First [Nothing, Nothing]     --
Nothing
getFirst $ foldMap Last [Just 1, Just 2]        -- Just
2
```

```
---- Basic
All True                -- Constructor
All False
All { getAll=True }     -- Same. All wrappers have a
single get* property
getAll $ All True       -- True


-- Boolean
All True
x = All True <> All True    -- All True <>=&&
getAll x                    -- True
x = Any True <> Any False   -- Any True <>=||
getAny x

-- Numeric
x = Sum 1 <> Sum 2          -- Sum 3
getSum x
x = Product 1 <> Product 2  -- Product 2
getProduct x

-- Maybe
-- First: returns the first Just
x = First Nothing <> First (Just 1) <> First (Just 2)
     -- First (Just 1)
getFirst x
-- Last: returns the last Just
x = Last (Just 1) <> Last (Nothing) <> Last (Just 2)
      -- Last (Just 2)
getLast x
```